

AQI Predictor

Real-time Air Quality Forecasting System

Test Documentation

Document Type:	Test Plan, Test Cases & Test Report
Version:	1.0
Date:	April 27, 2026
Author:	Charu Mittal
Framework:	pytest 7.x + FastAPI TestClient
Total Tests:	85
Result:	85 PASSED / 0 FAILED

Backend: FastAPI + SQLite + MLflow + Prometheus
Orchestration: Apache Airflow — Monitoring: Grafana

Contents

1	Test Plan	2
1.1	Scope	2
1.2	Test Objectives	2
1.3	Test Types	2
1.4	Test Files and Coverage	3
1.5	Tools and Environment	3
1.6	Test Execution Command	3
1.7	Pass / Fail Criteria	4
2	Test Cases	5
2.1	TC-DB — Database Service (<code>test_db_service.py</code>)	5
2.2	TC-H / TC-R — Health & Readiness (<code>test_health.py</code>)	5
2.3	TC-P — Predict Endpoint (<code>test_predict.py</code>)	5
2.4	TC-MI — Model Info Endpoint (<code>test_model_info.py</code>)	6
2.5	TC-MN / TC-MT / TC-MP — Model Service Helpers (<code>test_model_service.py</code>)	6
2.6	TC-DR — Drift Detection (<code>test_drift_detection.py</code>)	7
3	Test Execution Report	8
3.1	Executive Summary	8
3.2	Results by Module	8
3.3	Issues Encountered and Resolved	8
3.4	Warnings	8
3.5	Raw pytest Output	9
3.6	Coverage Summary	11

1. Test Plan

1.1. Scope

This test plan covers all automated unit and integration tests for the AQI Predictor FastAPI backend. The following areas are **out of scope** and are validated manually:

- Streamlit frontend UI behaviour
- Apache Airflow DAG execution and scheduling
- Docker Compose infrastructure and networking
- MLflow model training and experiment tracking
- Live OpenAQ / Open-Meteo API calls

1.2. Test Objectives

1. All API endpoints return correct HTTP status codes and Pydantic-validated response schemas.
2. SQLite database read, write, and upsert operations behave correctly under all conditions.
3. Model service helper functions handle edge cases: **None**, **NaN**, negative values.
4. Drift detection statistical logic (PSI, KS-test, Z-score) produces correct outputs.
5. No test carries a hard dependency on any running external service (MLflow, OpenAQ, DB file on disk).
6. All 85 tests pass with exit code 0 in under 60 seconds.

1.3. Test Types

Type	Description	Tool
Unit	Isolated function/class logic. No HTTP calls, no I/O, no running services.	pytest
Integration	FastAPI endpoints tested via <code>TestClient</code> . All external services are mocked with <code>MagicMock</code> .	pytest + <code>fastapi.testclient</code>

1.4. Test Files and Coverage

File	Module Tested	TC Range	Count	Type
test_db_service.py	db_service.py — SQLite reads/writes	TC-DB-01-07	7	Unit
test_health.py	/health, /ready endpoints	TC-H-01-03, TC-R-01-03	6	Integration
test_predict.py	/predict endpoint, all paths	TC-P-01-09	9	Integration
test_model_info.py	/model- info end- point	TC-MI-01-05	5	Integration
test_model_service.py	safe_number, safe_timestamp, predict_next_24h	TC-MN/MT/MP	12	Unit
test_drift_detection.py	compute_drift, PSI, severity, retrain flag	TC-DR-01-08	8	Unit
Additional files	Advisory, cities, feature service, ingestion, bootstrap	Various	38	Mixed
Total			85	

1.5. Tools and Environment

Tool / Component	Version / Purpose
pytest	7.x — primary test runner
fastapi.testclient	HTTP endpoint testing without a live server
unittest.mock	patch, MagicMock — service mocking
SQLite (tmp_path)	Real in-process DB via pytest tmp_path fixture
Python	3.11
Operating System	Ubuntu 22.04

1.6. Test Execution Command

```
cd backend
pytest tests/ -v --tb=short 2>&1 | tee metrics/test_report.txt
```

1.7. Pass / Fail Criteria

Criterion	Requirement
Exit code	Must be 0
Pass rate	100% — all 85 tests must pass
External dependencies	None — no running MLflow, OpenAQ, or DB file required
Max duration	Under 60 seconds
Warnings	Deprecation warnings allowed; must not cause test failures

2. Test Cases

Total: 85 — Unit: 47 — Integration: 38 — All Expected Result: PASS

2.1. TC-DB — Database Service (`test_db_service.py`)

Tests SQLite logic directly with real in-memory fixtures. No app imports, no mocking.

ID	Class	Description	Expected Result
TC-DB-01	TestGetRecentReadings	Only rows within time window returned	<code>len(rows) >= 46</code>
TC-DB-02	TestGetRecentReadings	Rows belong to queried city only	<code>All rows city == "Chennai"</code>
TC-DB-03	TestGetRecentReadings	Unknown city returns empty list	<code>rows == []</code>
TC-DB-04	TestGetLatestReading	Most recent row returned	<code>row is not None</code>
TC-DB-05	TestGetLatestReading	No row for unknown city	<code>row is None</code>
TC-DB-06	TestStorePrediction	Stored prediction retrievable with correct values	<code>current_aqi == 87</code>
TC-DB-07	TestStorePrediction	Same key upserts — old row replaced	<code>COUNT(*) == 1</code>

2.2. TC-H / TC-R — Health & Readiness (`test_health.py`)

ID	Class	Description	Expected Result
TC-H-01	TestHealthEndpoint	<code>/health</code> returns HTTP 200 + <code>{"status": "ok"}</code>	<code>status_code == 200</code>
TC-H-02	TestHealthEndpoint	Response has exactly the <code>status</code> key	<code>"status" in data</code>
TC-H-03	TestHealthEndpoint	<code>/health</code> responds under 500ms	<code>elapsed < 0.5s</code>
TC-R-01	TestReadyEndpoint	<code>/ready</code> returns <code>status=ready</code> when model loaded	<code>data["status"] == "ready"</code>
TC-R-02	TestReadyEndpoint	<code>/ready</code> returns <code>status=degraded</code> when not loaded	<code>data["status"] == "degraded"</code>
TC-R-03	TestReadyEndpoint	Response has <code>status</code> and <code>model_loaded</code> keys	Both keys present

2.3. TC-P — Predict Endpoint (`test_predict.py`)

ID	Class	Description	Expected Result
TC-P-01	TestPredictEndpointSuccess	Valid city + location_id returns HTTP 200	<code>status_code == 200</code>
TC-P-02	TestPredictEndpointSuccess	Response has <code>forecast_24h</code> with 24 values	<code>len == 24</code>
TC-P-03	TestPredictEndpointSuccess	Response has non-empty advisory string	<code>len(advisory) > 0</code>

ID	Class	Description	Expected Result
TC-P-04	TestPredictEndpointSuccess	Pollutants dict has all 6 keys	PM2_5, PM10, NO2, SO2, CO, O3
TC-P-05	TestPredictEndpointSuccess	Response city matches request	<code>data["city"] == "Chennai"</code>
TC-P-06	TestPredictEndpointErrors	Unsupported city returns HTTP 400	<code>status_code == 400</code>
TC-P-07	TestPredictEndpointErrors	Missing <code>location_id</code> returns HTTP 422	<code>status_code == 422</code>
TC-P-08	TestPredictEndpointErrors	Insufficient live rows returns HTTP 503	<code>status_code == 503</code>
TC-P-09	TestPredictEndpointErrors	Model inference failure returns HTTP 500	<code>status_code == 500</code>

2.4. TC-MI — Model Info Endpoint (`test_model_info.py`)

ID	Description	Expected Result
TC-MI-01	<code>/model-info</code> returns HTTP 200	<code>status_code == 200</code>
TC-MI-02	Response has <code>model_name</code> field	<code>"model_name" in data</code>
TC-MI-03	<code>supported_cities_count</code> matches fixture	<code>== 5</code>
TC-MI-04	<code>forecast_horizon_hours</code> is positive int	<code>isinstance(..., int) and > 0</code>
TC-MI-05	<code>supported_cities</code> is a list	<code>isinstance(..., list)</code>

2.5. TC-MN / TC-MT / TC-MP — Model Service Helpers (`test_model_service.py`)

ID	Class	Description	Expected Result
TC-MN-01	TestSafeNumber	<code>None</code> input returns 0.0	<code>== 0.0</code>
TC-MN-02	TestSafeNumber	<code>float('nan')</code> returns 0.0	<code>== 0.0</code>
TC-MN-03	TestSafeNumber	Integer 42 cast to 42.0	<code>== 42.0</code>
TC-MN-04	TestSafeNumber	Float 3.14 returned as-is	<code>approx(3.14)</code>
TC-MN-05	TestSafeNumber	Non-numeric string returns 0.0	<code>== 0.0</code>
TC-MT-01	TestSafeTimestamp	Missing key returns empty string	<code>== ""</code>
TC-MT-02	TestSafeTimestamp	Valid ISO string returned unchanged	<code>== "2026-04-26T10:00:00Z"</code>
TC-MT-03	TestSafeTimestamp	Explicit <code>None</code> value returns ""	<code>== ""</code>
TC-MP-01	TestPredictNext24h	<code>forecast_24h</code> has exactly 24 values	<code>len == 24</code>
TC-MP-02	TestPredictNext24h	Negative outputs clamped to 0	<code>All == 0</code>
TC-MP-03	TestPredictNext24h	Pollutant dict has 6 expected keys	PM25, PM10, NO2, SO2, CO, O3
TC-MP-04	TestPredictNext24h	<code>primary_pollutant</code> is a valid label	One of known pollutant set

2.6. TC-DR — Drift Detection (`test_drift_detection.py`)

ID	Description	Expected Result
TC-DR-01	Identical windows — no features drifted	<code>features_with_drift == 0</code>
TC-DR-02	500-unit mean shift triggers drift	<code>features_with_drift > 0</code>
TC-DR-03	Stable data — retrain not recommended	<code>retrain_recommended is False</code>
TC-DR-04	>40% features drifted — retrain recommended	<code>retrain_recommended is True</code>
TC-DR-05	Every feature dict has 5 required keys	<code>psi, ks_pvalue, z_score, severity, drift</code>
TC-DR-06	Severity is one of 4 valid levels	One of NONE, LOW, MODERATE, HIGH
TC-DR-07	<10 rows per column — feature skipped	<code>total_features_checked == 0</code>
TC-DR-08	<code>drift_fraction</code> always in <code>[0.0, 1.0]</code>	<code>0.0 <= frac <= 1.0</code>

3. Test Execution Report

3.1. Executive Summary

Metric	Value
Total Test Cases	85
Passed	85
Failed	0
Skipped	0
Warnings	9 (non-blocking deprecation only)
Duration	3.65 seconds
Exit Code	0
Overall Result	ALL TESTS PASSED

3.2. Results by Module

Module	Type	Total	Passed	Failed
test_db_service	Unit	7	7	0
test_health	Integration	6	6	0
test_predict	Integration	9	9	0
test_model_info	Integration	5	5	0
test_model_service	Unit	12	12	0
test_drift_detection	Unit	8	8	0
Additional test files	Mixed	38	38	0
Total		85	85	0

3.3. Issues Encountered and Resolved

Issue	Root Cause	Fix Applied
TC-R-02 failed on first run	<code>is_model_ready</code> patched at service module, not at router's import reference	Changed patch target to <code>app.routers.health.is_model_ready</code>
DB test schema mismatch	Fixture DDL used camelCase column names (<code>cityreadings</code> , <code>currentaqi</code>)	Rewrote fixture to match actual schema (<code>city_readings</code> , <code>current_aqi</code>)
Drift test not discovered	File named <code>tets_drift_detection.py</code> (typo)	Renamed to <code>test_drift_detection.py</code>

3.4. Warnings

All 9 warnings are of type:

```
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
```

for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: `datetime.datetime.now(datetime.UTC)`.

These appear only in `test_db_service.py` fixtures. They are **non-blocking** and do not affect test results. The fix is to replace `datetime.utcnow()` with `datetime.now(timezone.utc)` in fixtures.

3.5. Raw pytest Output

```
===== test
    session starts
    =====
platform linux -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0
rootdir: /home/charu-mittal/AQI_Predictor
plugins: hydra-core-1.3.2, anyio-4.13.0, typeguard-4.5.1
collected 85 items

tests/test_advisory_service.py .....

    [ 21%]
tests/test_cities.py ....

    [ 25%]
tests/test_db_service.py .....

    [ 34%]
tests/test_drift_detection.py .....

    [ 50%]
tests/test_health.py .....

    [ 57%]
tests/test_model_info.py .....

    [ 63%]
tests/test_model_service.py .....

    [ 77%]
tests/test_openaq_service.py .....

    [ 89%]
tests/test_predict.py .....

    [100%]

=====
    warnings summary
    =====
tests/test_db_service.py::TestGetRecentReadings::test_returns_rows_within_window
tests/test_db_service.py::TestGetRecentReadings::test_returns_correct_city
tests/test_db_service.py::TestGetRecentReadings::test_no_rows_for_unknown_city
```

```

tests/test_db_service.py::TestGetLatestReading::test_returns_most_recent_row
tests/test_db_service.py::TestGetLatestReading::test_returns_none_for_unknown_city
/home/charu-mittal/AQI_Predictor/tests/test_db_service.py:77:
  DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
  for removal in a future version. Use timezone-aware objects to represent
  datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.utcnow()

tests/test_db_service.py::TestGetRecentReadings::test_returns_rows_within_window
/home/charu-mittal/AQI_Predictor/tests/test_db_service.py:101:
  DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
  for removal in a future version. Use timezone-aware objects to represent
  datetimes in UTC: datetime.datetime.now(datetime.UTC).
  since = (datetime.utcnow() - timedelta(hours=48)).isoformat()

tests/test_db_service.py::TestGetRecentReadings::test_returns_correct_city
/home/charu-mittal/AQI_Predictor/tests/test_db_service.py:113:
  DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
  for removal in a future version. Use timezone-aware objects to represent
  datetimes in UTC: datetime.datetime.now(datetime.UTC).
  since = (datetime.utcnow() - timedelta(hours=200)).isoformat()

tests/test_db_service.py::TestStorePrediction::test_store_and_retrieve_prediction
/home/charu-mittal/AQI_Predictor/tests/test_db_service.py:169:
  DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
  for removal in a future version. Use timezone-aware objects to represent
  datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.utcnow().isoformat()

tests/test_db_service.py::TestStorePrediction::test_duplicate_prediction_upserted
/home/charu-mittal/AQI_Predictor/tests/test_db_service.py:187:
  DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
  for removal in a future version. Use timezone-aware objects to represent
  datetimes in UTC: datetime.datetime.now(datetime.UTC).
  ts = datetime.utcnow().isoformat()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 85 passed,
  9 warnings in 3.57s
=====

```

3.6. Coverage Summary

Functional Area	Test IDs	Status
SQLite DB reads, writes, updates	TC-DB-01 to TC-DB-07	PASS
Health & readiness endpoints	TC-H-01-03, TC-R-01-03	PASS
Predict endpoint — happy path	TC-P-01 to TC-P-05	PASS
Predict endpoint — error handling	TC-P-06 to TC-P-09	PASS
Model info endpoint	TC-MI-01 to TC-MI-05	PASS
Model helper functions	TC-MN-01-05, TC-MT-01-03, TC-MP-01-04	PASS
Drift detection logic	TC-DR-01 to TC-DR-08	PASS
Additional modules	38 cases	PASS